



Building our own

MCP Server

using Python

The "USB Port" for AI Agents



Chandra Sekhar

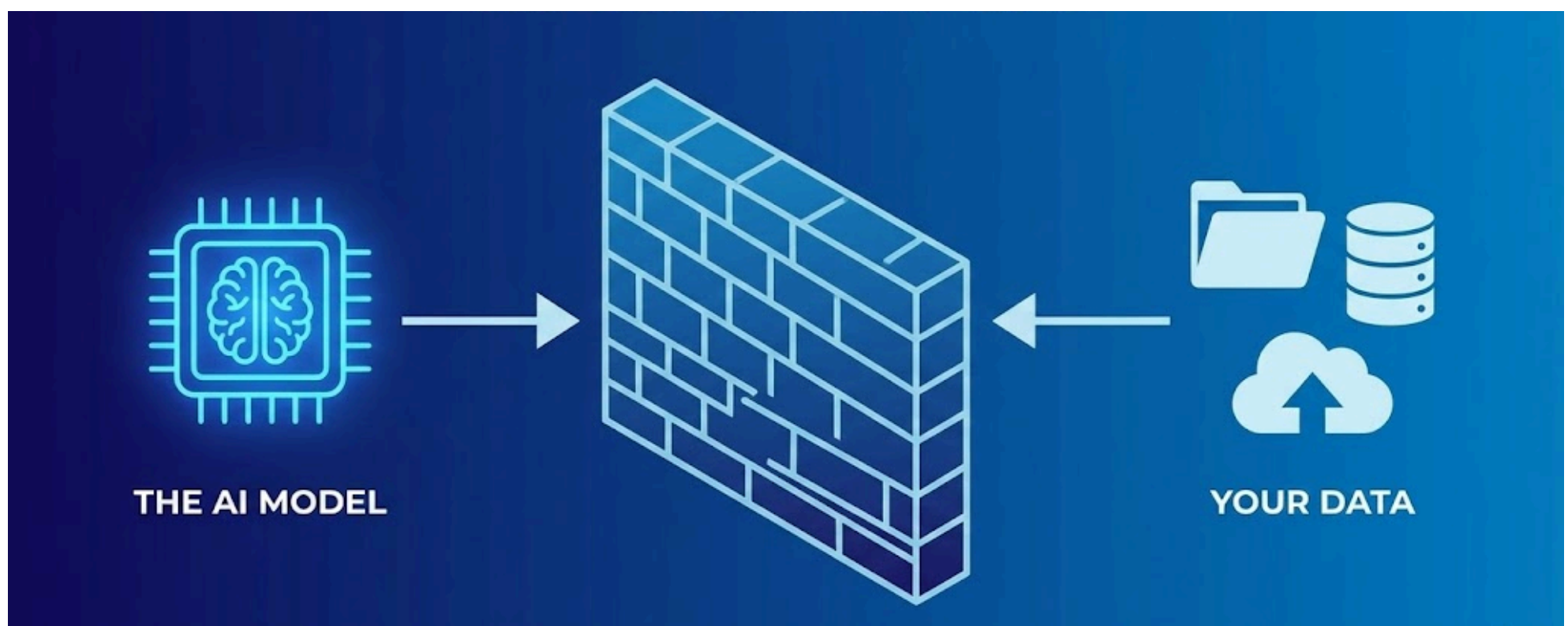
AI Engineer & Trainer

The Context Gap

AI models (Claude, GPT) are smart, but they don't have access to your:

- Local Files
- Internal APIs
- Live System Data

They only know what you type in the prompt.



Enter MCP

- MCP is an open standard that lets AI models talk to your systems securely.
- Think of it as an API specifically designed for AI Assistants.
- The Goal: Give the AI tools to fetch its own data.



Let's Build our own MCP (Python)

Install Necessary Modules

- We will use FastMCP to simplify the process.
- Prerequisites: Python 3.9+

```
pip install fastmcp requests
```

Create the Server

```
from mcp.server.fastmcp import FastMCP  
  
mcp = FastMCP("demo-mcp-server")
```



Create a "Tool"

- Use the `@mcp.tool()` decorator.
- This turns a standard Python function into a skill the AI can use.

```
@mcp.tool()
def add(a: int, b: int) → int:
    """Add two numbers"""
    return a + b
```

Connect to the World

- Let's give the AI access to live weather data using requests.
- Now the AI isn't hallucinating weather; it's fetching it.




```
@mcp.tool()
def get_weather(city: str) → str:
    """Get current weather"""
    url = f"https://wttr.in/{city}"
    return requests.get(url).text
```

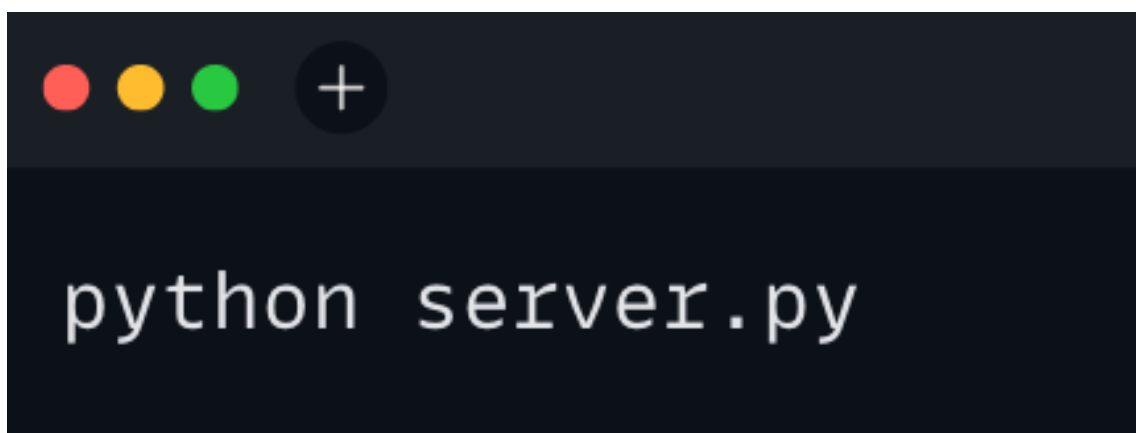
Running the Server

- Start the server using Server-Sent Events (SSE).
- The server listens for the AI's requests and streams the data back.

```
if __name__ == "__main__":
    mcp.run(transport="sse")
```



You can now run the server from your terminal:

A terminal window with a dark background and a title bar containing three colored circles (red, yellow, green) and a plus sign. The text 'python server.py' is displayed in a light gray font.

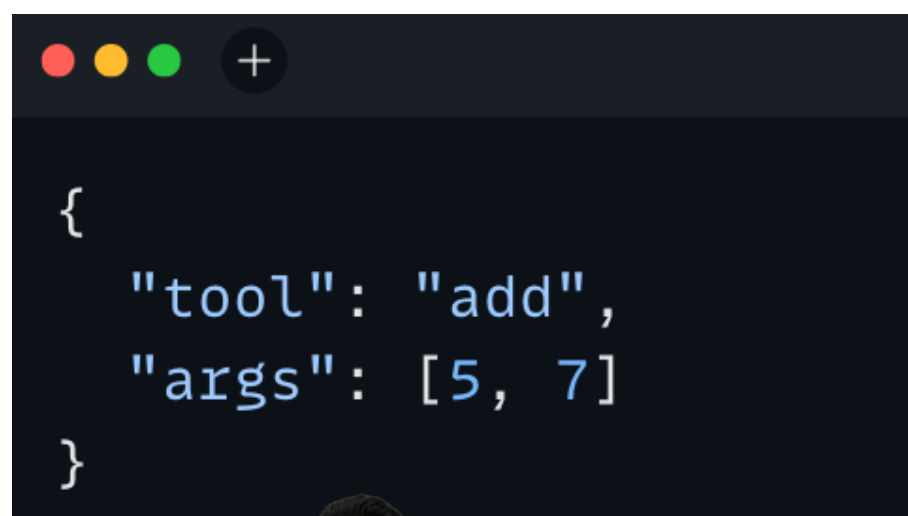
```
python server.py
```

Your MCP server is now live and ready to accept requests.

Testing the Tools

To test your tools, you need an MCP-compatible client such as ChatGPT with developer features or another app that supports the protocol. Once connected, the client will list your available tools.

For example, you can send a request like this:

A terminal window with a dark background and a title bar containing three colored circles (red, yellow, green) and a plus sign. A JSON object is displayed in a light gray font.

```
{  
  "tool": "add",  
  "args": [5, 7]  
}
```



Want more content like this?



Tap that follow button and
stay in the loop!



Like



Comment



Share



Save